Models and Tools for Cyber-Physical Systems

Chapter 3: Safety Requirements

Instructors: Suman Sourav, Jonathan Julian Huerta y Munive
{sumansourav, jjhymuni}@cs.aau.dk

Original slides courtesy of Rajeev Alur
alur@cis.upenn.edu

Predicates

- A predicate (or property) is a function that returns true or false:
  - $P: X \rightarrow \{0, 1\}$

- Examples of predicates
  - On a finite domain: if $X = \{x_1, x_2\}$ and if $P(x_1) = 1$ and $P(x_2) = 0$ then P is a predicate
  - On an infinite domain: if X represents the integers then "is greater than 10" is a predicate
  - In a day-to-day CPS situation: if X is the state space of a multithreaded system, then "shares the same resources as thread1" is a predicate

Requirements

- Requirements: Collection of (desirable) properties about a designed system such that, if all of them are satisfied, then the system is deemed acceptable

- If system S satisfies all its requirements, we also say it "meets all requirements"

- Example of possible requirements for software:
  - "respects privacy laws"
  - "is safe"
  - "irrespective of the combination of inputs, the output is within the range x_{min} and x_{max}"
  - "it never deadlocks"
  - "$\forall s. \forall t.\ |s(x,t) - s(y,t)| > \theta$"

Imprecise

More precise

Formal

Informal

Requirements

- High-assurance/safety-critical systems requires high precision:
  - You can refine "is safe" to a collection of several requirements of the form
    - "if the system starts satisfying condition P, then it satisfies Q throughout/at-the-end-of its execution"
    - "in all possible executions of the system, at some point P holds"

- Procure to have a clear separation between requirements (what needs to be implemented) and system (how it is implemented)

- Verification problem: Given a requirement φ and a system/model C, prove/disprove that the system/model C satisfies the requirement φ

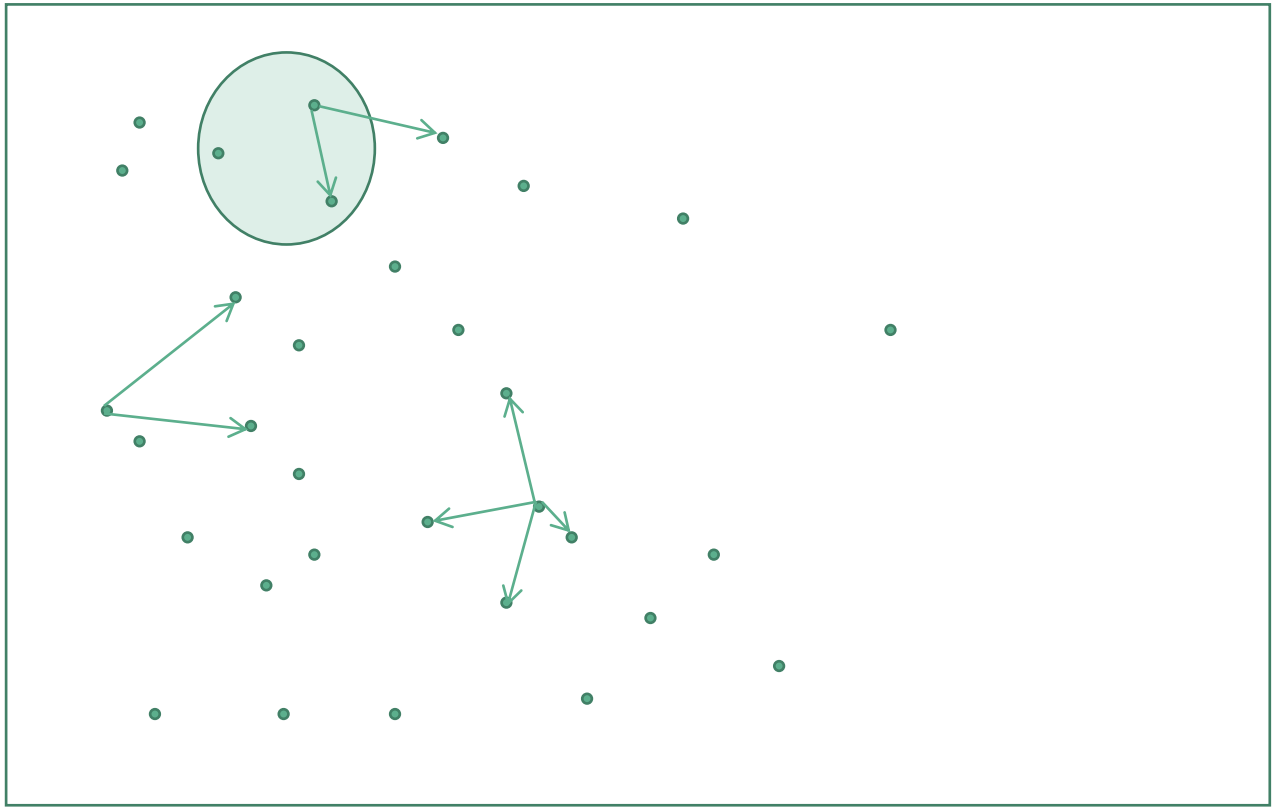
Safety and Liveness Requirements

- A safety requirement states that a system always stays within "good" states (i.e., "nothing bad ever happens")
  - Cruise controller: Desired speed stays between bounds
  - Collision avoidance: Distance between two cars is always greater than some minimum threshold
- A liveness requirement states that a system eventually gets to a goal state (i.e., "something good eventually happens")
  - Cruise controller: Actual speed eventually equals desired speed
  - Multithreading: Each thread eventually terminates
- Formalization and analysis techniques for safety and liveness differ significantly; we focus on safety

Transition Systems and Reachable States

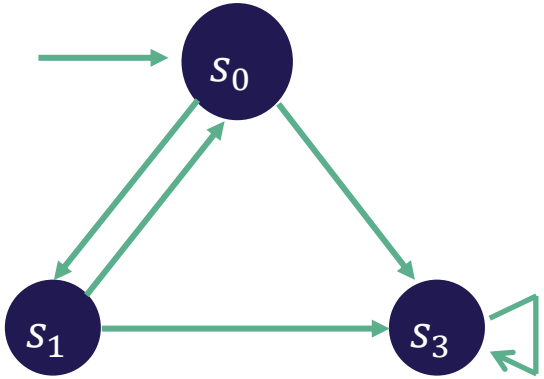
Transition Systems

States + Initial states + Transitions between states



Definition of Transition System

- A transition system T = (S, Init, Trans) consists of
  1. A set S of states
  2. A binary relation (the transition description Trans) to update state variables
  3. Initialization Init for state variables
- Transition systems are often depicted as directed graphs:



$$S = \{s_0, s_1, s_3\}$$

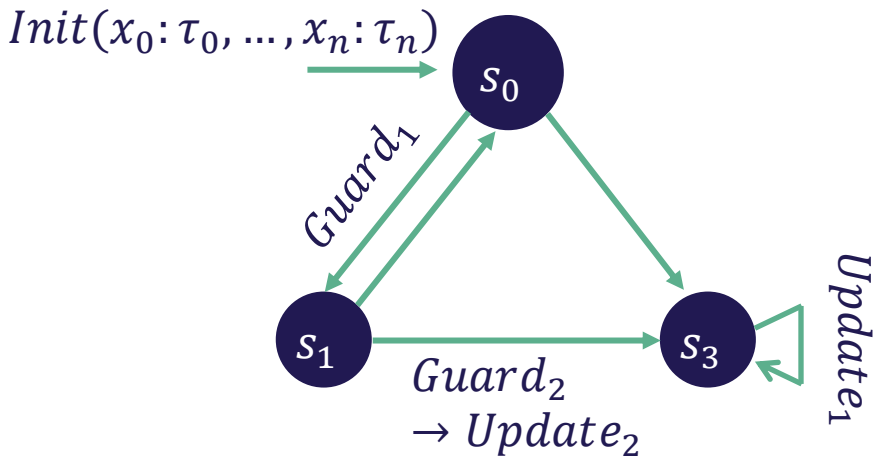
$$\rightarrow = \{(s_0, s_1), (s_0, s_3), (s_1, s_0), (s_1, s_3), (s_3, s_3)\}$$

$$S_0 = \{s_0\}$$

- Most discrete systems (e.g., synchronous reactive components, programs, …) can be represented as a transition system

Extended-state machine

- A transition system extended with (typed) state variables and labels (called mode-switches) for the transitions predicating over the state variables
- Abstract representation of an extended-state machine:
  - Can only transition when guards hold
  - Transitions occur while updating the state variables according to the updating rule



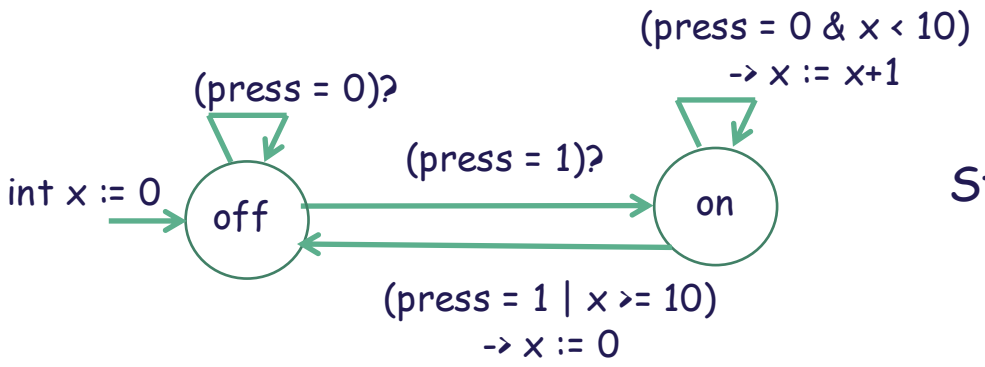
$$S = \{s_0, s_1, s_3\}$$

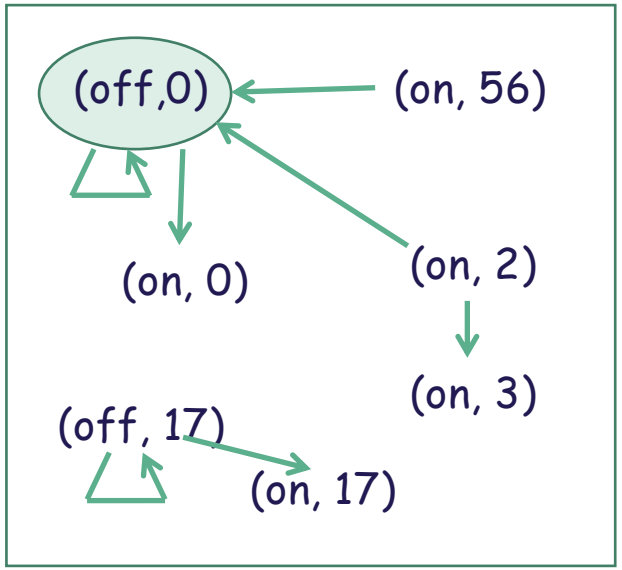
$$S_0 = \{s_0\}$$

$$Vars = \{x_0: \tau_0, \dots, x_n: \tau_n\}$$

$$L = \{\{(s_0, s_1) \mapsto Guard_1\}, \{(s_3, s_3) \mapsto Update_1\}, \{(s_1, s_3) \mapsto Guard_2 \rightarrow Update_2\}\}$$

"Switch" Extended state machine





State variables:

{off, on} mode, int x

Initialization:

mode = off; x = 0

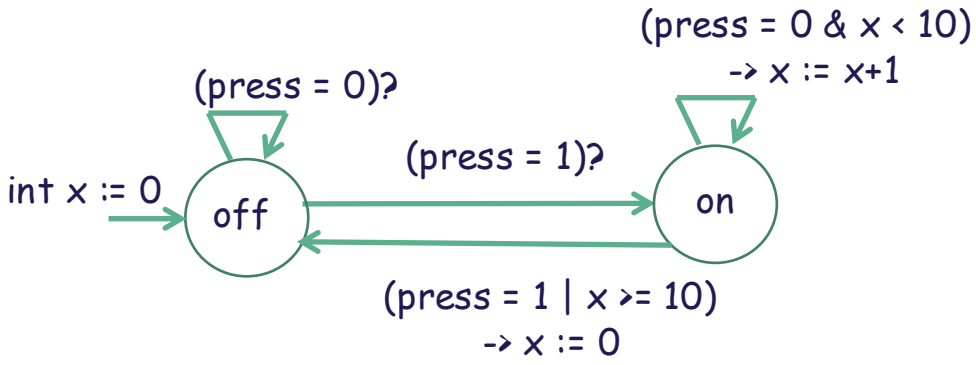
Transitions:

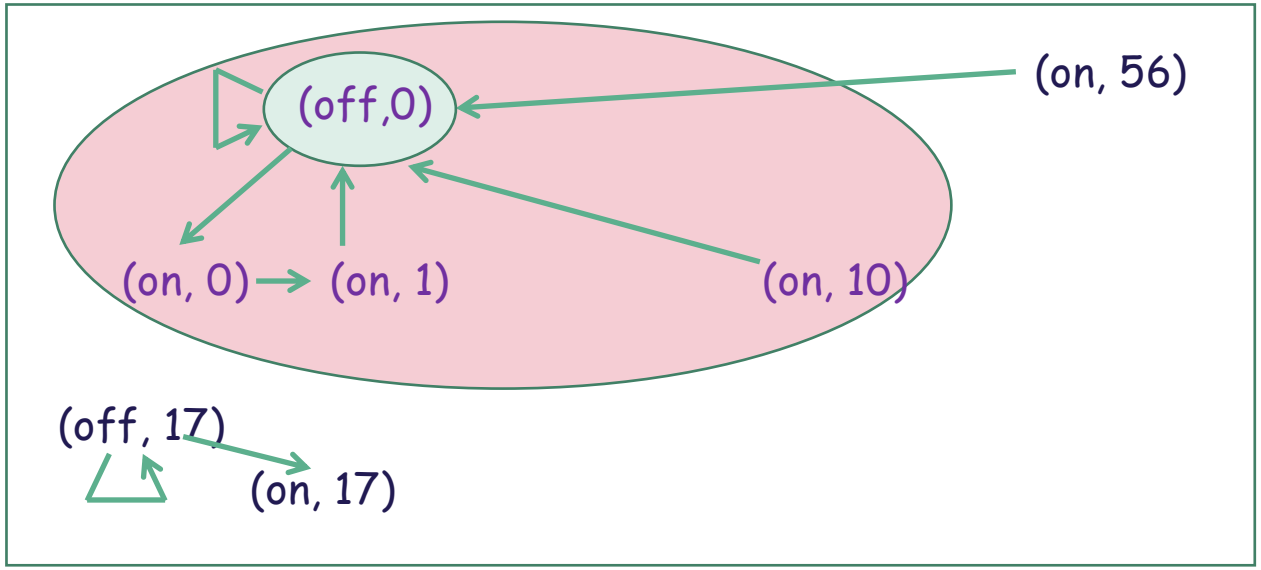
(off,n) -> (off,n);
(off,n) -> (on,n);
(on,n) -> (on,n+1) if n<10;
(on,n) -> (off,0)

Input/output variables become local variables

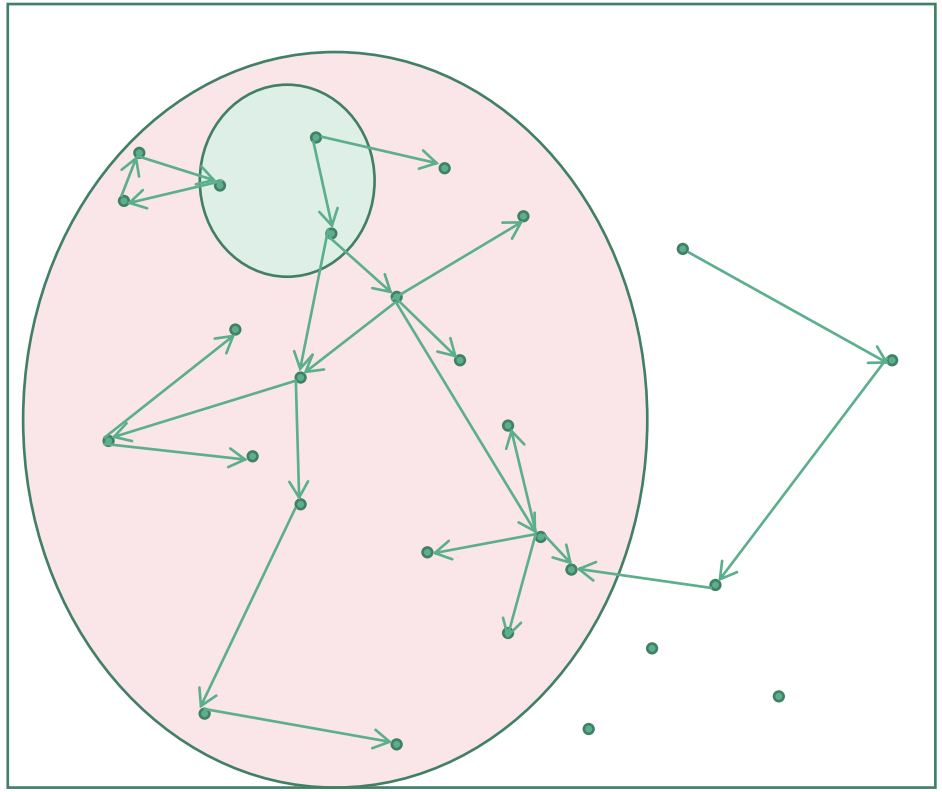
Values for input variables chosen nondeterministically

Reachable States





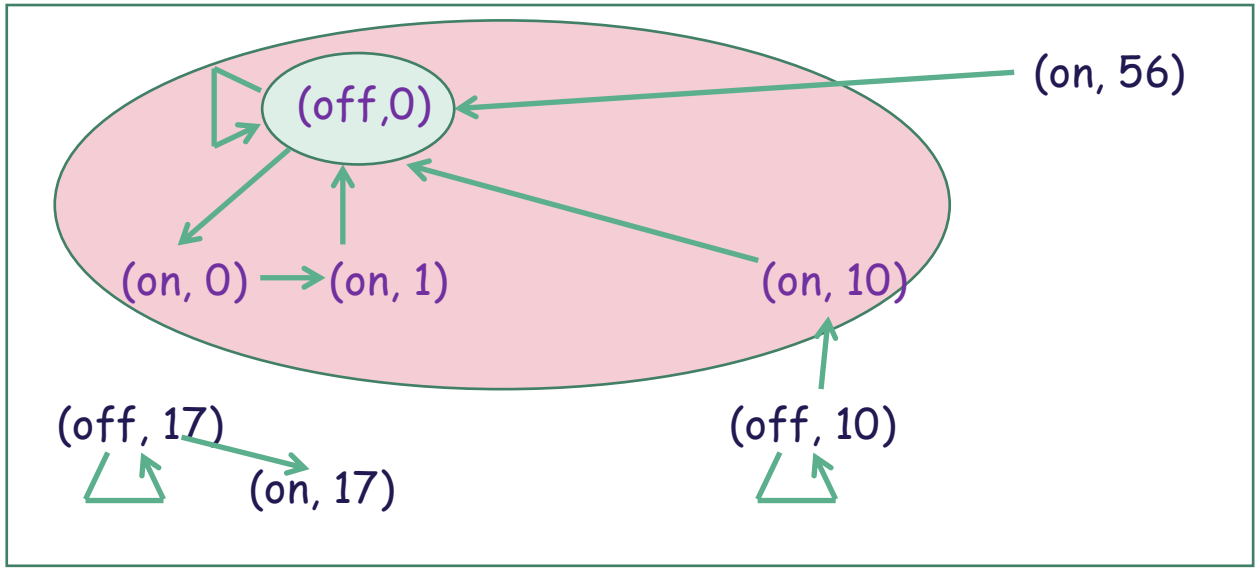
Reachable States of Transition Systems



A state s of a transition system is reachable if there is an execution starting in some initial state and ending in the state s

Invariants

Invariants



- For the moment, we only consider simple properties of transition systems expressed as Boolean-valued expressions φ over state variables
- Property φ is an invariant of T if every reachable state satisfies φ
- Examples of invariants: x <= 10; x <= 50; mode = off -> x = 0
- Examples of properties that are not invariants: x < 10; mode = off

Invariants for Safety Requirements

- Invariants are safety properties
- Express desired safety requirement as property φ of state variables
  - If φ is an invariant, then the system is safe
  - If φ is not an invariant, then some state satisfying $\sim\varphi$ is reachable (and an execution leading to such a state is a counterexample)

- Invariants can express all state-based safety requirements
  - Example for non-state-based requirement: "The button should be pressed at most three times" (unless the number of "button pressed" can be read from the state)
  - We will see how to deal with general safety requirements later

Dynamic vs. Static Analysis

- Dynamic analysis (runtime)
  - Execute the system multiple times with different inputs
  - Check if every execution meets the desired requirement
- Static analysis (design time)
  - Analyze the source code or the model for possible bugs
- Trade-offs
  - Dynamic analysis is incomplete, but accurate (checks real system, bugs discovered are real bugs)
  - Static analysis can catch design bugs early
  - Static analysis is not always scalable (solution: analyze approximate versions, which can lead to false warnings)

Invariant Verification

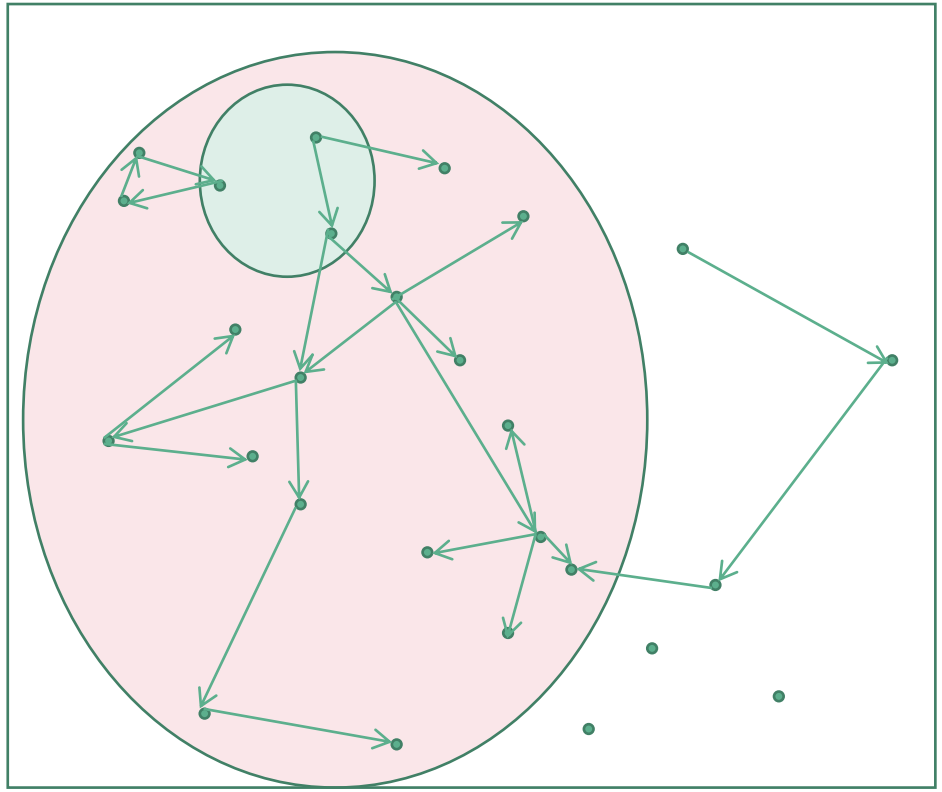
- Simulation
  - Simulate the system multiple times with different inputs
  - Easy to implement, scalable, but no correctness guarantees
- Proof-based (Theorem proving)
  - Construct a proof that the invariant (property) holds
  - May require manual effort (partial automation possible)
- State-space analysis (Model checking)
  - Algorithm explores "all" reachable states to check invariants
  - Not scalable in theory, but tools can analyze many real-world designs

Recap: Inductive Proof Over Natural Numbers

- Show that a statement P holds for all natural numbers n
  - Base case: Prove that P holds for n = 0
  - Assume that P holds for some natural number n = k
  - Prove (using the assumption) that the statement holds for n = k+1

- Benefit: can prove properties over an infinite domain

Reachable States are Inductive



A state s of a transition system is reachable if there is an execution starting in an initial state and ending in the state s

Proving Invariants

- Inductive definition of reachable states
  - All initial states are reachable using 0 transitions
  - If a state s is reachable in k transitions and (s, t) is a transition, and the state t was not reachable in fewer steps, then t is reachable in k+1 transitions
  - Reachable = Reachable in n transitions, for some n
- Given a transition system T and a property φ, prove that φ is an invariant, i.e., that all reachable states of T satisfy φ
- Prove: for all n, states reachable in n transitions satisfy φ
  - Base case: Show that all initial states satisfy φ
  - Inductive step: Show that if a (reachable?) state s satisfies φ and (s, t) is a transition, then t must satisfy φ

Inductive Invariant

- A property φ is an inductive invariant of transition system T if
  1. Every initial state of T satisfies φ
  2. If a state s satisfies φ and (s, t) is a transition of T, then t must satisfy φ

- Note that we dropped the restriction that s is reachable

- **Theorem:** If φ is an inductive invariant, then all reachable states of T must satisfy φ, and thus, φ is an invariant

Proving Inductive Invariant: Example (1)

- Consider a transition system T parametric in m given by
  - State variable int x, initialized to 0
  - Transitions given by "if (x < m) then x := x+1"
- Is the property φ : 0 <= x <= m an inductive invariant of T ?
- Base case: Consider initial state (x = 0). Check that it satisfies φ
- Inductive step:
  - Consider an arbitrary state s, suppose s(x) = a
  - Assume that s satisfies φ, i.e., assume 0 <= a <= m
  - Consider the state t obtained by executing the transition from s
  - If a < m, then t(x) = a+1, otherwise t(x) = a
  - In either case, 0 <= t(x) <= m
  - So t satisfies the property φ, and the proof is complete
- Thus φ is an inductive invariant

Proving Inductive Invariant: Example (2)

- Consider transition system T given by
  - State variables int x, y; x is initially 0, y is initially m
  - Transitions given by "if (x < m) then {x := x+1; y := y-1}"
- Is the property φ : 0 <= y <= m an inductive invariant of T ?
- Base case: Consider initial state (x = 0, y = m). Check that it satisfies φ
- Inductive step:
  - Consider an arbitrary state s with x = a and y = b
  - Assume that s satisfies φ, i.e., assume 0 <= b <= m
  - Consider the state t obtained by executing the transition from s
  - If a < m, then t(y) = b-1, otherwise t(y) = b
  - Can we conclude that 0 <= t(y) <= m ?
  - No! The proof fails! In fact, φ is not an inductive invariant of T!

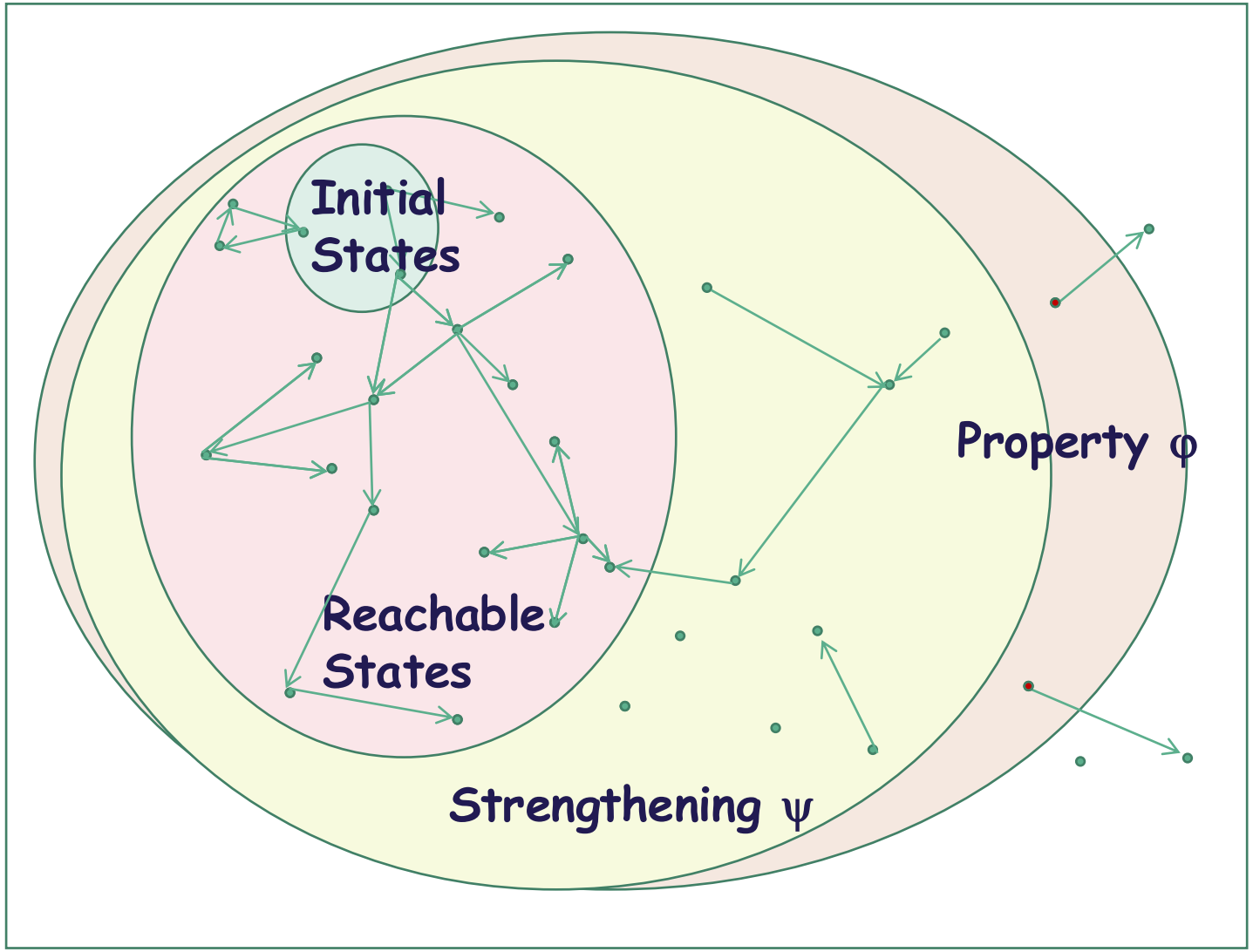
Why did the proof fail?

- Consider the state s = (x = 0, y = 0)
  - State s satisfies φ: 0 <= y <= m
  - Executing transition from s leads to state t = (x = 1, y = -1)
  - State t does not satisfy φ !
- However, the state s in the above argument is not reachable!
- Cause of failure: Property φ did not capture correlation between variables x and y
- Solution: Inductive strengthening
  - Consider property ψ: (0 <= y <= m) & (x+y = m)
  - Property ψ implies property φ
  - While φ is not an inductive invariant, ψ is (see next slide)
  - It follows that all reachable states must also satisfy φ

Proving Inductive Invariant: Example (3)

- Consider transition system T given by
  - State variables int x, y; x is initially 0, y is initially m
  - Transitions given by "if (x < m) then {x := x+1; y := y-1}"
- Property ψ : (0 <= y <= m) & (x+y = m)
- Base case: Consider initial state (x = 0, y = m). Check that it satisfies ψ
- Inductive step:
  - Consider an arbitrary state s with x = a and y = b
  - Assume that s satisfies ψ, i.e., assume 0 <= b <= m and a+b = m
  - Consider the state t obtained by executing a transition from s
  - If a < m then t(x) = a+1 and t(y) = b-1, otherwise t(x) = a and t(y) = b
  - But if a < m, since a+b = m holds, b > 0, and thus b-1 >= 0
  - In either case, the condition 0 <= t(y) <= m & t(x)+t(y) = m holds!
- Conclusion: Property ψ is an inductive invariant!

Inductive Invariants



Proof Rule for Proving Invariants (1)

- Recall:
  Theorem: If φ is an inductive invariant, then all reachable states of T must satisfy φ, and thus, φ is an invariant

- Say we want to prove that φ is an invariant
  If we can prove that φ is an inductive invariant, the theorem implies that φ is an invariant

- However, the converse of the theorem does generally not hold
  - Not every invariant is inductive, so φ may not be inductive
- Good news: If φ is indeed an invariant, there is always an inductive strengthening ψ of φ that is an inductive invariant – why?
  - Pick ψ as "set of reachable states" (which satisfies φ by definition)
- Bad news: The set of reachable states (ψ) cannot generally be computed

Proof Rule for Proving Invariants (2)

- Goal: establish that a property φ is an invariant of transition system T

- Approach: find another property ψ such that
  1. ψ implies φ (that is, a state satisfying ψ must satisfy φ)
     - If you view φ and ψ as sets, then ψ is a subset of φ
  2. ψ is an inductive invariant
     - Show that every initial state satisfies ψ
     - Assume that a state s satisfies ψ. Consider any state t such that (s, t) is a transition. Show that t satisfies ψ

Summary of Invariants

- General-purpose proof technique for proving invariants of programs/models/systems
- Inductive invariant:
  - Holds in initial states
  - Preserved by every transition/reaction
- To be inductive, a property typically needs to capture the relevant relationship among all state variables
- Benefit of inductive invariants:
  - Correctness reasoning becomes local (only need to think about what happens in one step)
  - Tools available to check whether a property is an inductive invariant
- Area of active research: can we automatically find inductive invariants?

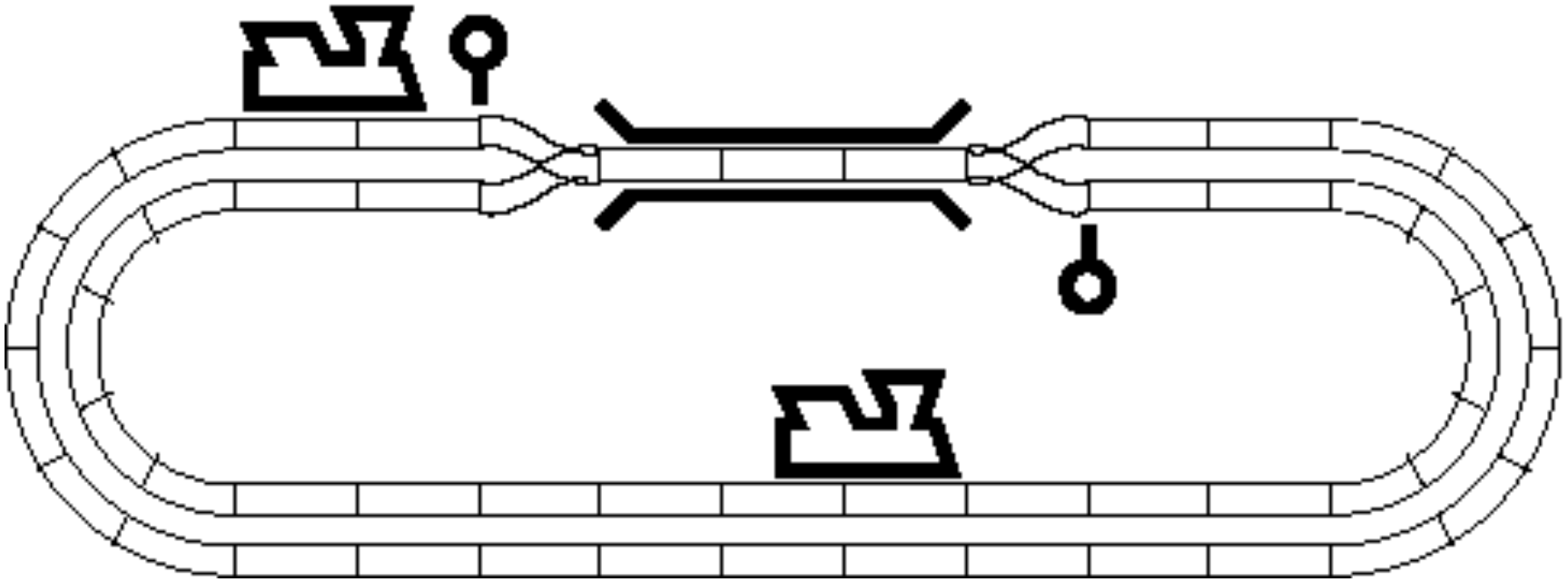
Case Study: Railroad Controller

Requirements-Based Design

- Systematic approach to design of systems
- Given:
  - Input/output interface of system C to be designed
  - Model E of the environment
  - Safety property φ of the composite system

- Design problem: Fill in details of C (state variables, initialization, and update) so that C || E satisfies the invariant φ

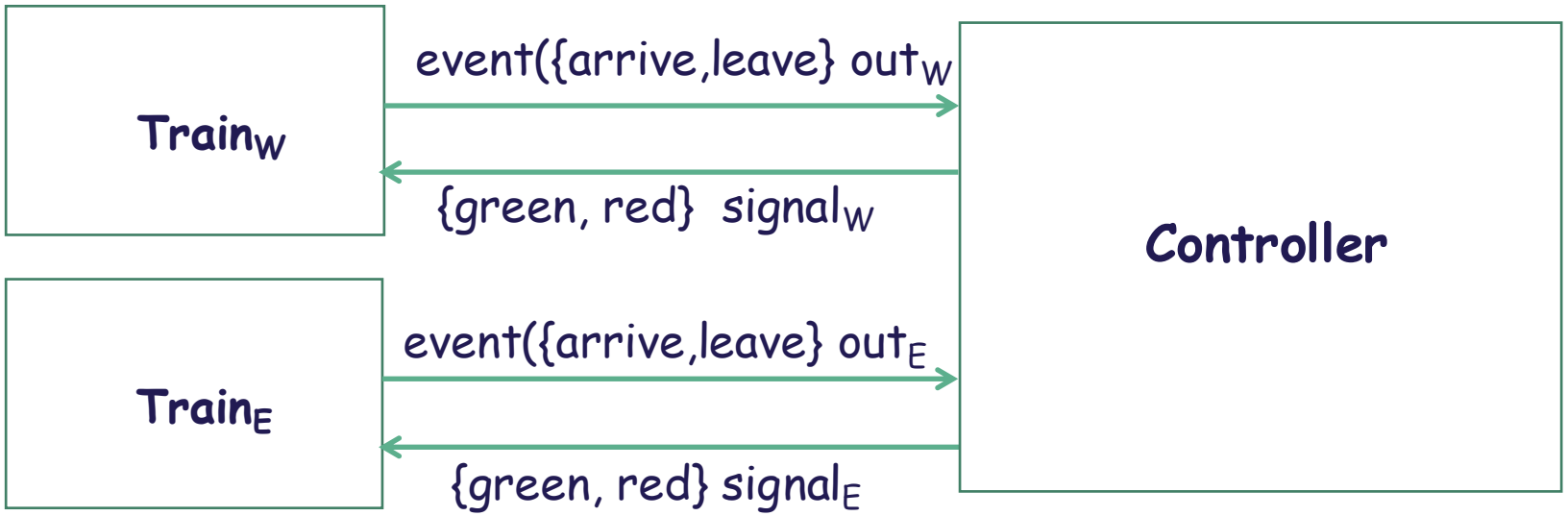
Railroad Controller Example



Train Model

- From the perspective of the controller, the train is initially (far) away
- The train can be away for an arbitrarily long period
- When the train gets close, it communicates with the controller via an event arrive, and now it is in a different state wait
- In the wait state, the train monitors the signal:
  - If the signal is green, the train enters the bridge (state bridge)
  - If the signal is red, the train continues to wait
- A train can stay on the bridge for a duration that is not exactly known (and not influenced by the controller)
- When the train leaves the bridge, it communicates with the controller via an event leave, and goes back to the away state
- This behavior repeats: a train that is away may again request entry
- Both trains have symmetric behavior

Controller Design Problem

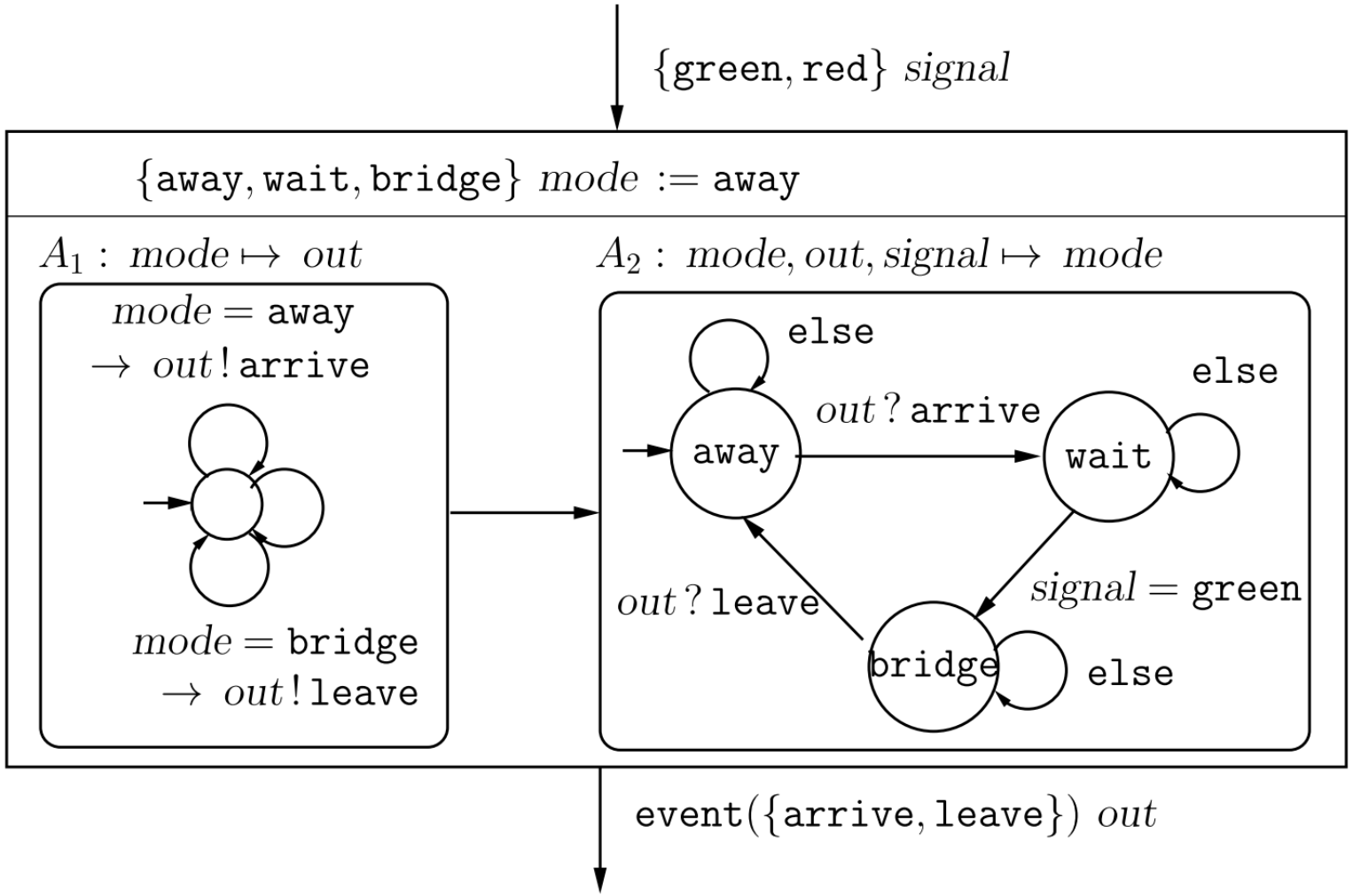


Safety requirement: Trains are not on the bridge simultaneously

$$\sim(\text{mode}_W = \text{bridge} \ \& \ \text{mode}_E = \text{bridge})$$

This is an invariant requirement

Synchronous Component "Train"

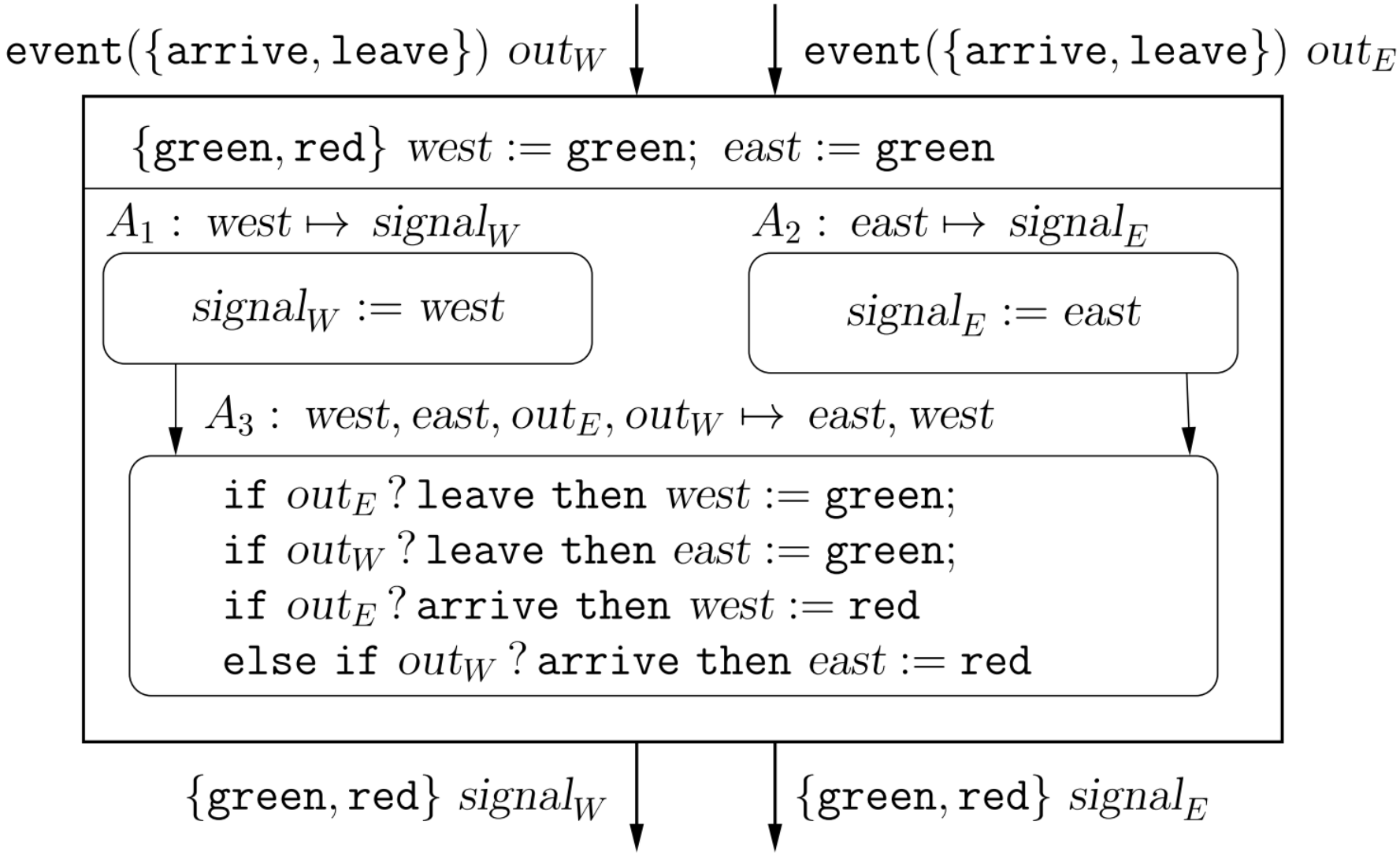


First Attempt at Controller Design

- Controller maintains two state variables to track the state of each signal
- Both state variables are initially green
- Set the output variables based on the corresponding state variables
- If a train arrives, then update the opposite signal variable to red to block the other train from entering
- If a train leaves, reset the opposite signal variable to green

- What happens if both trains arrive simultaneously?
  - Tie breaker: Give priority to east train

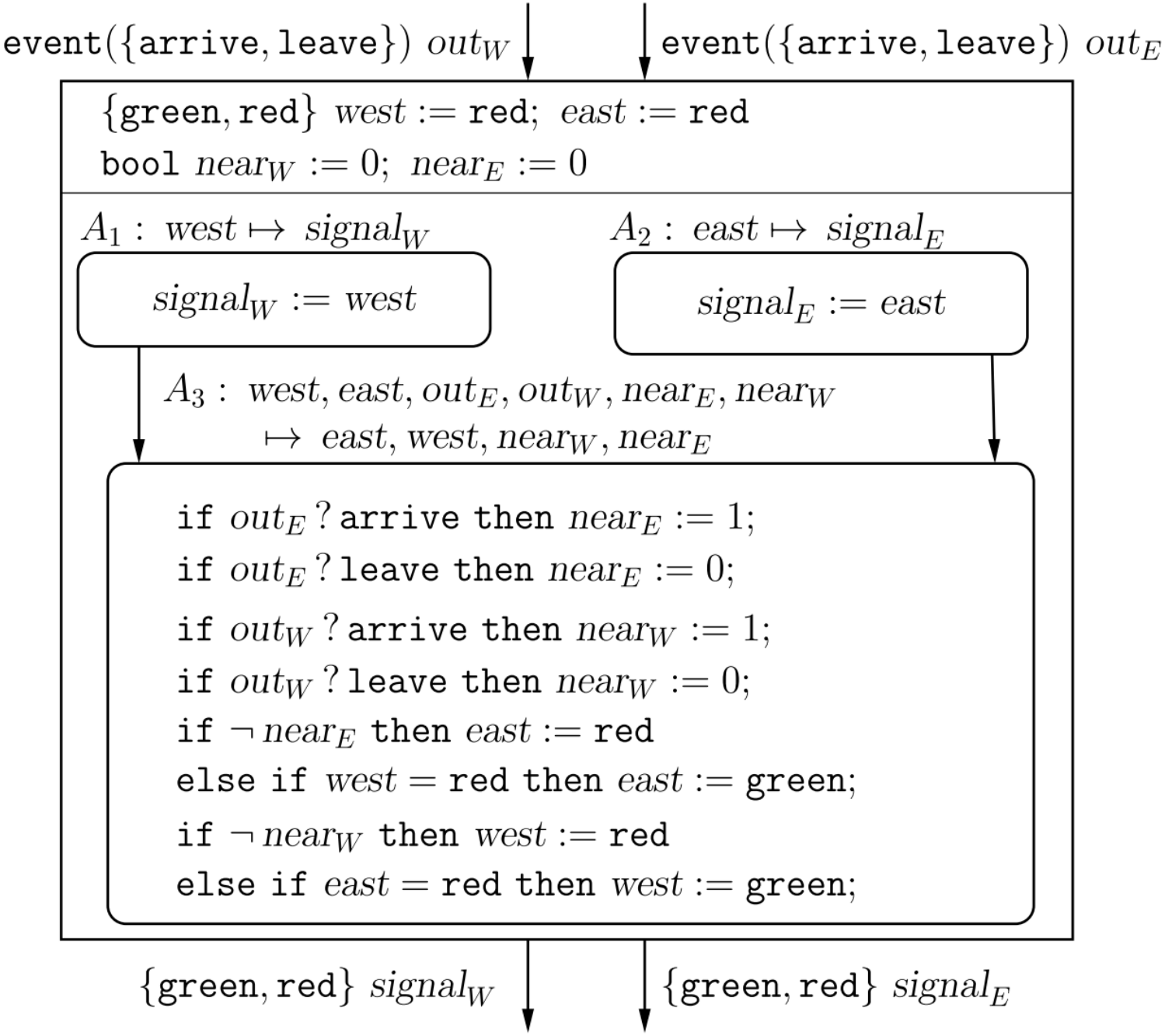
Synchronous Component "Controller1"



west	east	$mode_W$	$mode_E$
green	green	away	away
↓ green	↓ green	↓ arrive!	↓ arrive!
red	green	wait	wait
↓ red	↓ green	↓	↓
red	green	wait	bridge
↓ red	↓ green	↓	↓ leave!
green	green	wait	away
↓ green	↓ green	↓	↓ arrive!
red	green	bridge	wait
↓ red	↓ green	↓	↓
red	green	bridge	bridge

Second Attempt at Controller Design

- What went wrong the first time? Controller did not remember whether a train was waiting at each entrance
- We fix this by adding a new Boolean variable $near_W$ that remembers whether the west train wants to use the bridge
  - Is initially 0
  - Is changed to 1 when the west train issues arrive
  - Is reset back to 0 when the west train issues leave
- Invariant: $mode_W$ = away if and only if $near_W = 0$
- We add another,symmetric variable $near_E$
- We also now keep both signals red by default
- A signal is changed to green if the corresponding train is near and the other signal is red, and changed back to red when the train is away
- Still resolve simultaneous arrivals by preferring the east train


event({arrive, leave}) out_W
event({arrive, leave}) out_E
{green, red} $west$:= red; $east$:= red
bool $near_W$:= 0; $near_E$:= 0
A_1 : $west \mapsto signal_W$
$signal_W$:= $west$
A_2 : $east \mapsto signal_E$
$signal_E$:= $east$
A_3 : $west, east, out_E, out_W, near_E, near_W$
$\mapsto east, west, near_W, near_E$
if out_E ? arrive then $near_E$:= 1;
if out_E ? leave then $near_E$:= 0;
if out_W ? arrive then $near_W$:= 1;
if out_W ? leave then $near_W$:= 0;
if $\neg near_E$ then $east$:= red
else if $west$ = red then $east$:= green;
if $\neg near_W$ then $west$:= red
else if $east$ = red then $west$:= green;
{green, red} $signal_W$
{green, red} $signal_E$

Properties of Controller2

- The system RailRoadSystem2 = Controller2 || TrainW || TrainE satisfies the safety requirement (invariant)
- Additional (alternative) requirements we could be interested in
  1. If the west train is waiting, the west signal will eventually become green
  2. If the west train is waiting for its signal to turn green, the other train should not be allowed on the bridge more than once
- Requirement 1 is a liveness property (see Chapter 4)
- Requirement 2 is a safety property: its violation can be demonstrated by a (finite) execution in which the east train enters, leaves, and enters again while the west train keeps waiting with its signal red
  - Not an invariant (not expressible over state variables only)